

Random Features, GIN, and Why Degree Wins

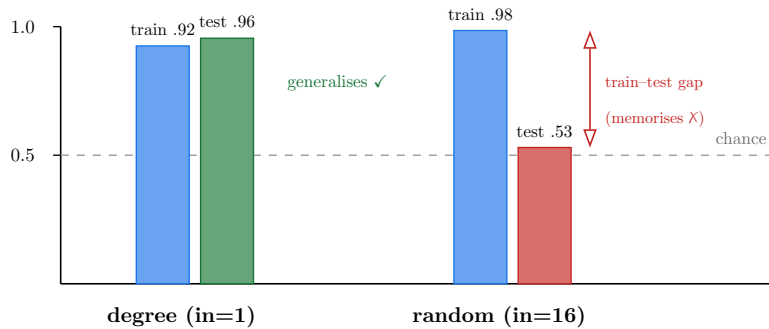
what message passing learns on `connectedness_hard` — and what it only memorises

`src/layers.py` (GINConv) · `src/dataset.py` · 2026-06-24

The puzzle. We expected random node features (rGIN) to help an mpGNN reason about connectivity and plain degree to be useless. The runs said the opposite: 3-layer GIN reached **97.5% test** with **degree**, but only **53% test** with **random** features — while **memorising** the training set (98.5% train). This note explains GIN, what random features actually do, why degree generalises here while random does not — and why even degree’s win is probably a local shortcut, not connectivity reasoning.

1 The two results

Both runs: 3 GIN layers, hidden 64, mean pooling, 10k graphs (8k train / 2k test), identical except the node feature.

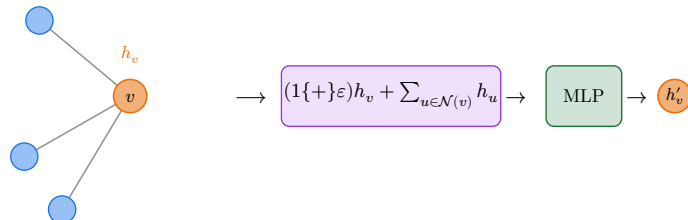


The gap is the whole story: with **random** features the model fits the training set almost perfectly (98.5%) but tests at chance — a textbook generalisation failure. With **degree** train and test rise together and stay close. To see why, we need GIN and what each feature feeds it.

2 How GIN works

A **Graph Isomorphism Network** layer updates every node from its own value plus the **sum** of its neighbours’ values, then passes the result through a small MLP:

$$h_v^{(k)} = \text{MLP}^{(k)} \left((1 + \varepsilon) \cdot h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right).$$



Two design choices matter:

Sum, not mean or max summation preserves **multiplicity** — “three neighbours valued 1” differs from “one neighbour valued 1”. This is what makes GIN as discriminative as the **Weisfeiler–Lehman** graph-isomorphism test (Xu et al. 2019); mean and max throw multiplicity away and so cannot tell some structures apart.

An MLP after aggregation a learnable nonlinearity over the aggregated multiset, so the layer can compute rich functions of a neighbourhood, not just a linear average.

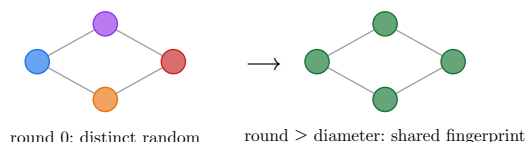
After k layers a node’s vector summarises its **k -hop neighbourhood**. Crucially, information travels exactly **one hop per layer** — remember this for §5.

What the feature feeds in. GIN computes functions of the **node features it is given**. With $h_v^{(0)} = \text{degree}$, it builds functions of the degree pattern around each node. With $h_v^{(0)} = \text{a random vector}$, it builds functions of a random pattern. Same machinery, completely different raw material — that is the whole experiment.

3 How random node features work

Plain GINs are blind to symmetry: two structurally-identical nodes (same neighbourhood up to isomorphism) get the **same** features forever, so the network literally cannot tell them apart. On `connectedness_hard` the two blobs are near-symmetric, so constant or matched-degree features give the model almost nothing to distinguish nodes.

Random node initialisation (rGIN; Sato et al. 2021, Abboud et al. 2021) breaks the symmetry by hand: give every node an independent random vector. Now distinct nodes are distinct, and sum-aggregation can build a **reachable-set fingerprint** — a node accumulates the (random) signatures of everyone it can reach:



Nodes in the **same component** converge to the **same** fingerprint (they reach the same set); nodes in **different** components keep different fingerprints. In principle that is exactly the connectivity signal — which is why we tried it.

3.1 The catch: expressiveness is not generalisation

Random features make the network **more expressive** — but expressive power cuts both ways. With 16 random dimensions and sum-aggregation, the model can hand every training graph a **near-unique fingerprint** and simply memorise “this fingerprint → this label”. That is precisely what happened: **98.5% train, 53% test**. The random values carry **no signal that transfers** — a test graph arrives with brand-new random numbers the model has never seen, so its memorised lookup is useless and it falls back to chance.

Our `node_features`: `random` samples the vectors **once per load** (fixed for the run). That is the most memorisable setting: the model sees the same random pattern for each training graph every epoch. The stronger rGIN variant **resamples every forward pass**, which denies the model any stable values to memorise and forces it to use randomness only as a symmetry-breaker. Worth trying — but it does not change the deeper point below.

4 Why degree generalises and random does not

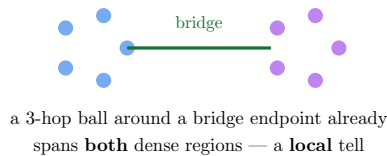
	degree	random
what it is	a real structural quantity, shared across all graphs	per-graph noise, different every graph
dimensionality	1 — tiny capacity to memorise	16 — ample capacity to memorise
transfers to test?	yes: the same structural pattern recurs	no: test graphs have unseen values

	degree	random
result	train .92 / test .96	train .98 / test .53

Generalisation needs the feature to carry signal that is **the same kind of thing** in train and test. Degree does: a degree-pattern that distinguishes the classes in training graphs distinguishes it in test graphs too. Random features do not: whatever the model learns about specific random values is meaningless on fresh draws. High capacity + no transferable signal = **memorise the train set, fail the test set**. This is the same shortcut-vs-signal lesson as the rest of these experiments, seen from the feature side.

5 The twist: degree’s win is (probably) a local shortcut

Before crowning degree, look at the depth. The connected-class **diameter is 4–8** (median 5), but degree succeeded with only **3 GIN layers** — a 3-hop receptive field. Information cannot cross a diameter-8 graph in 3 hops, so the model **cannot** be tracing global reachability. It must be reading something **local**.



In `connectedness_hard` the classes differ by exactly one edge: a **bridge** between the blobs (connected) versus an extra edge **inside** a blob (disconnected). A bridge endpoint’s few-hop neighbourhood looks different from an internal node’s — two dense regions stitched together vs. one — and because the blobs are dense, that difference shows up **within 3 hops**. So GIN+degree learns to **detect the bridge motif locally**, which on this dataset happens to decide connectivity — without any global reasoning.

Consequence for the thesis. `connectedness_hard` killed the obvious degree-statistic shortcut (min/mean degree are matched), but it did **not** kill a subtler **local-structure** shortcut: a few-hop GIN can spot the bridge. Degree’s 97.5% is real generalisation, but most likely of a **local pattern**, not of connectivity **reasoning**. The clean way to tell them apart:

- **Depth ablation:** if 2–3 hops already suffice (they do), the model is local — true reachability would need depth $\geq$ diameter.
- **Size generalisation:** train on $n \leq 20$, test on $n = 40\text{--}50$. A local bridge-detector keeps working (the motif is size-independent); a model that truly needed global reachability would degrade. Strong transfer here would **confirm** it is the local shortcut.

6 Summary

Concept	What to remember
GIN	sum-aggregate neighbours + self, then MLP; sum keeps multiplicity (1-WL power); 1 hop per layer
random features	break node symmetry so MP can build reachable-set fingerprints; add expressive power
expressive $\neq$ general	16-dim random features let GIN memorise each graph (train .98) but carry no transferable signal (test .53)
why degree wins	a low-dim, shared structural feature transfers train→test; the model learns a recurring pattern, not noise

Concept	What to remember
the catch	3 hops < diameter, yet 97.5% — so degree detects the bridge locally ; a shortcut, not global connectivity reasoning
how to check	depth ablation + train-small/test-large size generalisation

The headline is counter-intuitive but clean: **more expressive features made the model worse**, because expressiveness without transferable signal is just a licence to memorise — and the humble degree feature won by carrying real, reusable structure. Whether that structure amounts to **reasoning** or merely **local bridge-spotting** is the next thing to pin down.